

Pick-and-Place DSL

A DSL the LLM emits to drive a pick-and-place robot. The LLM is the only stateful component. Every value the next invocation needs appears in the current invocation's output; the system keeps the most recent of each block in context plus turn history.

1. Execution model

1.1 Vocabulary

- **Conversation:** Collection of turns. Is interrupted by user (starting new conversation). scans, position, gripper, held carry across turns.
- **Turn:** one user prompt and the model's response. May span many invocations.
- **Invocation:** one LLM forward pass.
- **Step:** one numbered unit in PROGRAM. One step = one invocation.
- **This turn:** after the last user message
- **Location** is a single slot in the scene, can only contain one object at most
- **Location Collection** is name of a group of one or more Locations
- **Target** is a **Location** or multiple **Location** or **Location Collection** or multiple **Location Collection**

1.2 Context manager and Blocks.

There are messages of only two roles: ASSISTANT and USER. Note, I don't use typical tool message role, instead write tool results as special blocks into assistant messages.

For ASSISTANT messages, there is no freeform text, everything is in some block. There are blocks, that the model generates and that can be passed to the model as input context: PRECALL MAP STATE PROGRAM RESOLUTION TOOLS OUTCOME CHECK ERROR ANSWER in this exact order. And there are two blocks that are tool return values, they can be passed to the model: PRECALL RESULTS TOOLS RESULTS, not generated by it.

The **Context manager** is a program that parses and keeps track of the blocks and user messages and passes the right input to the model based on conditions described below. It NEVER parses the content of the blocks. It also makes tool calls, one by one, updating tools results after each tool call and invoking the model. Updating some block counts even if the content isn't changed, if it was generated by the model, its an update, even if the content was empty and stays empty.

These blocks may be empty: MAP TOOLS CHECK ERROR PRECALL RESULTS TOOLS RESULTS. All the other blocks cannot be empty. If model generates one of them, then with content.

1.3 Input combinations

Blocks are added in the order shown. For blocks, if not stated explicitly otherwise, it is meant, that

the model receives the most recent state of block, not all previous.

#	Input
I1	all previous pairs role USER message - ANSWER block + most recent role USER message + if PRECALL PRECALL RESULTS were generated this conversation: PRECALL PRECALL RESULTS
I2	TOOLS TOOLS RESULTS or PRECALL PRECALL RESULTS, depends on what was generated last
I3	if OUTCOME was generated this turn: all previous pairs role USER message - ANSWER block + most recent role USER message + if PRECALL PRECALL RESULTS were generated this conversation: PRECALL PRECALL RESULTS + MAP STATE PROGRAM + if TOOLS TOOLS RESULTS were generated this turn: TOOLS TOOLS RESULTS + (if ERROR was generated this turn: ERROR, elif OUTCOME was generated this turn: OUTCOME)

1.4 Output combinations

Exactly one of these per invocation. Blocks are emitted in the order shown.

#	Output
O1	PRECALL
O2	MAP STATE `` PROGRAM RESOLUTION TOOLS optional: OUTCOME
O3	CHECK
O4	ERROR
O5	STATE RESOLUTION TOOLS optional: OUTCOME
O6	STATE ANSWER

1.5 System Loop

Here is how **Context manager** decides what type of input to give to the model. Reaching **O6** stops the loop and ends the turn.

Input	Possible Outputs (Model decides)
I1	O1 or O2
I2	O3 or O4

Input	Possible Outputs (Model decides)
I3	O5 or O6

Previous Output	Condition	Input
O1	-	I2
O2	if T00LS is not empty	I2
O2	if T00LS is empty	I3
O3	if previous input this turn not counting O3 was O1	I1
O3	if previous output this turn not counting O3 was O2 or O5	I3
O4	-	I3
O5	if T00LS is not empty	I2
O5	if T00LS is empty	I3

Explanation:

1. The model can make precalls that last during whole conversation **I1, O1**
2. After that the model generates mapping and program + usual state, resolution and tool calls: **O2**.
3. **I2** is tool error handling step. Both precalls and tool calls go through it.
4. **I3->O5->I2->O3->I3** is the most common program execution routine with tool calls.
5. **I3->O6** happens when **OUTCOME** or **ERROR** was generated and ends the program with **ANSWER**

1.6 Block format

Each block has the form:

```
BLOCK_NAME
<content>
```

The next block's header on its own line ends the previous block. An empty block is its header followed immediately by the next header.

2. PRECALL Block

This is essentially a tool call that only differs from **T00LS** blocks in that it is preserved and passed to the model across turns. The model uses it for tool calls, which results it needs to save. In my case it is `get_positions()`.

```
PRECALL
get_positions()
```

3. STATE block

```
STATE
cursor: step_<N> | answer_<K> | error
position: <Position> | unknown
gripper: open | closed | unknown
held: <ObjectType> | none | unknown
scans: { <Location>: <ObjectType>, ... }
visited: [<Position>, ...]
memory: { <Name>: <Value>, ... }
```

Field semantics:

- **scans**: cumulative conversation-scoped map of *occupied* slots observed so far. A location covered by some pose in **visited** but absent from **scans** is **known empty**.
- **visited**: observe poses executed in the current turn. Reset each turn.
- **memory**: program-defined variables. Cleared at turn boundaries.

3.1 Tool effects on STATE

Tool	Effect
move_to(P)	position := P
locate_shapes()	append current position to visited; merge returned slot → type into scans; remove from scans any slot covered by the current pose but absent from the return
open_gripper()	gripper := open, held := none
close_gripper()	gripper := closed; held := T from the current pick_<T>_<L> pose or from the most recent move_to tool call argument or from the user message (i.e. "Close the gripper, you're holding X) or unknown otherwise
get_gripper_state()	gripper := <return>; if open, held := none
get_position_state()	position := <return>

After a successful **pick** X from L: scans[L] is removed. After a successful **place** X at L: scans[L] := X.

The model applies these effects when rewriting STATE for the next invocation.

User-stated state changes ("you are holding a cube") are applied directly: the model writes the corresponding STATE update in the same invocation it emits the program.

3.2 Cursor advancement rules

1. Cursor starts with step_1 when the program is emitted

2. If cursor in the input shows at `step_<N-1>`, in the output it must show at `step_<N>`.
3. Regardless of the rule 2 cursor shows at `answer_<K>` or `error` if input contains block **OUTCOME** with content `answer_<K>` or block **ERROR**

4. MAP

Translates user-prompt phrases for objects and locations into scene-canonical names or lists thereof.

```
MAP
<phrase> = <value>
<phrase> = <value>
```

`<value>` is either a single canonical name or a bracketed list `[name1, name2, ...]`.

4.1 Generation

For every distinct content noun phrase in the user prompt referring to an object type or a location, emit one MAP line — even when the phrase already matches a canonical name. Identity MAPs are intentional and uniform:

```
MAP
the cube = red_cube
red_cube = red_cube
the second bin = box2
box1 = box1
any drawer = [drawer1, drawer2, drawer3]
```

4.2 Inferring values from the position list

The scene provides only the position list. The model parses entries to infer entity structure: from `pick_red_cube_drawer1`, `pick_red_cube_drawer2`, `pick_red_cube_drawer3`, the model infers locations `drawer1`, `drawer2`, `drawer3` and, for "any drawer", produces `[drawer1, drawer2, drawer3]`.

Position-list entries may use any naming convention (`pick_red_cube_drawer1`, `drawer1_pick_red_cube`, etc.). The model identifies pick / place / observe entries by the name only.

4.3 What is *not* mapped

- **Pronouns** ("it", "the one I'm holding") — programs read STATE directly: `held`.
- **Numbers, kinds** — used inline in conditions.
- **"Somewhere" / "anywhere" with no inferable group** — no MAP line.

4.4 PROGRAM uses MAP phrases

PROGRAM uses MAP phrases (the left-hand sides) only — never raw MAP values. Values are expanded at resolution time.

5. PROGRAM

5.1 What a program is

A PROGRAM is an ordered sequence of numbered steps emitted as the PROGRAM block:

```
PROGRAM
STEP 1: <step body>
STEP 2: <step body>
...
STEP N: <step body>
```

The rule is **one step = one invocation**.

5.2 Lines of a program

Every line inside PROGRAM is exactly one of the following kinds. The interpreter parses by line role; each role has a fixed shape.

Line kind	Shape
Step header	STEP <N>:
Primitive line	<primitive>
Branching arm head	when <cond>: otherwise when <cond>: otherwise:
Outcome line	answer_<K>

Indentation under a head line groups the primitives belonging to that head. The arm heads must each be followed by at least one indented primitive line or an outcome line.

5.2 Position-name pattern notation

`pick_<T>_<L>` and `place_<T>_<L>` denote position-list entries whose structure encodes "pick T at L" / "place T at L". The actual entry name is whatever the position list contains; the model finds it by structural match, not literal string construction.

5.3 Step kinds

A step body is exactly one of two kinds. The kind is determined by syntactic shape.

1. **Linear** — one or more primitive lines, no head. All primitives unwind into a single TOOLS batch.
2. **Branching** — one or more `when / otherwise when` arms, optionally followed by one `otherwise` arm. Conditions are evaluated in order; the first true arm wins. If none match and no `otherwise` is present, the step is `fallthrough` (no work; cursor advances). Arms inside of arms not allowed.

5.4 Position-name pattern notation

`pick_<T>_<L>`, `place_<T>_<L>`, and observe poses like `observe_<L>` denote entries in the position list whose structure encodes the action. The actual entry string is whatever the position

list contains; the LLM finds the matching entry by structural match on <T> and <L>

5.5 Primitives

Every primitive is described by four facts: its **syntax**, the **tool calls** it unwinds to, the **RESOLUTION** lines it emits, and any **precondition** on STATE.

<X>, <W> are always object types: either left hand sides in **MAP** or names derived from positions list.

<A>, is always target: either left hand sides in **MAP** or names derived from positions list.

<L> or <location> is always single location: either left hand sides in **MAP** or name derived from positions list.

<Position> must always be a valid position from the position list or a left hand side in the **MAP** that equals a valid position from the position list

5.5.1 Simple actions

Primitive	Tool calls	Resolution lines	Precondition
go to <Position>	move_to(<Position>)	(none)	—
open gripper	open_gripper()	(none)	—
close gripper	close_gripper()	(none)	—
check gripper	get_gripper_state()	(none)	—
check position	get_position_state()	(none)	—
locate here	locate_shapes()	(none)	—

5.5.2 Observe actions

Primitive	Tool calls	Resolution lines	Precondition
observe [<A1>, <A2>, ...] (drop [] if single argument)	For each observe pose P covering some location in <A1>, <A2>, ... and $P \notin \text{STATE.visited}$: move_to(P); locate_shapes().	observe poses = [P1, P2, ...]	
observe everything	For each observe pose in get_positions not in STATE.visited: move_to(P); locate_shapes().	observe poses = [P1, P2, ...]	

Primitive	Tool calls	Resolution lines	Precondition
observe next unvisited position	move_to(P); locate_shapes() next P in order of positions list with $P \notin \text{STATE.visited}$	chosen_pose = P	
observe next unvisited position not covering <A>	As above, with the constraint that P covers no location in <A>.	chosen_pose = P	

5.5.3 Pick and place actions

Primitive	Tool calls	Resolution lines	Precondition
approach pick of <X> at <A>	move_to(pick_<X>_ >_<L>)	(none beyond <X> / <A> helpers, if any)	scans[<L>] == <X>
pick from <A>	move_to(pick_<X>_ >_<L>); close_gripper()	L = <A> = <location>T = scans[<L>] = <type>	gripper open; <A> resolves to a single Location L; $L \in \text{STATE.scans}$
approach place of <X> at <A>	move_to(place_< X>_<L>)	(none beyond <X> / <A> helpers, if any)	<L> is known empty
place at <A>	move_to(place_< X>_<L>); open_gripper()	L = <A> = <location>X = held = <type>	gripper closed; STATE.held is an ObjectType (not none, not unknown); <A> resolves to a single Location L; L is known empty

5.5.4 Memory

Primitive	Effect	Resolution lines
remember <name> = <Expr>	After the step's tool calls run, resolve <Expr> against the post-tools STATE and write the value to STATE.memory[<name>].	<Expr> = <value> if <Expr> is a helper; nothing if <Expr> is a literal.

5.5.5 Outcome

Primitive	Effect	Resolution lines
answer_<K>	Emits the OUTCOME block with content answer_<K> in	(none)

Primitive	Effect	Resolution lines
	the current invocation. The step's other primitives still run. The next invocation is the answer invocation.	

5.6 ObjectType helpers

Each non-literal expression emits one RESOLUTION line of the form `<expression> = <value>`

Expression	Value
literal type name (from MAP)	itself
held	<code>STATE.held</code>
<code><memory_name></code>	<code>STATE.memory[<name>]</code>
a type in <code><A></code>	Let <code>L</code> = first occupied in <code><A></code> ; value is <code>scans[L]</code>
a type in <code><A></code> other than <code><W></code>	Let <code>L</code> be the first slot in <code><A></code> (slot order) with <code>scans[L] ≠ <W></code> ; value is <code>scans[L]</code>
a type shared by <code><A></code> and <code></code>	value is first <code>X</code> in (types in <code><A></code> , slot order) such that some slot of <code></code> has <code>scans[·] == X</code>
a type in <code><A></code> not in <code></code>	value is first <code>X</code> in (types in <code><A></code> , slot order) such that no slot of <code></code> has <code>scans[·] == X</code>

5.7 Location helpers

Each emits one RESOLUTION line of the form `<expression> = <value>` or `L/T = <expression> = <value>` in case of pick/place primitive expansion.

Expression	Value
literal Location name (from MAP)	itself; no resolution line
<code><memory_name></code> (storing a Location)	<code>STATE.memory[<name>][0]</code> ; emit <code><memory_name> = <value></code>
first empty in <code><A></code>	First <code>L</code> in <code><A></code> (slot order) that is covered by <code>STATE.visited</code> and absent from <code>STATE.scans</code>
first occupied in <code><A></code>	First <code>L</code> in <code><A></code> (slot order) present in <code>STATE.scans</code>
first location in <code><A></code> holding <code><X></code>	First <code>L</code> in <code><A></code> (slot order) with <code>scans[L] == <X></code>
first location holding <code><X></code>	First <code>L</code> anywhere in <code>STATE.scans</code> (insertion order of scans) with entry <code><X></code>

Expression	Value
first location holding <X> outside <A>	As above, excluding any L in <A>
first empty location	First L covered by STATE.visited and absent from STATE.scans (order: position-list order over all Locations)
first empty location outside <A>	As above, excluding any L in <A>

5.8 Conditions

Each evaluated condition emits one RESOLUTION line of the form <cond> = <value>. Value is bool unless stated otherwise

Robot.

Condition	Value
gripper is open	STATE.gripper == open
gripper is closed	STATE.gripper == closed
gripper is known	STATE.gripper != unknown
holding anything	STATE.held is an ObjectType (not none, not unknown)
holding <X>	STATE.held == <X>
holding is known	STATE.held != unknown
at home	STATE.position == home
at <Position>	STATE.position == <Position>
position	STATE.position (returns position name; used in comparisons)
visited all observe positions	exists no observe position not in STATE.visited

Targets

Condition	Value
<A> is empty	No $L \in \langle A \rangle$ is in STATE.scans
<A> is full	Every $L \in \langle A \rangle$ is in STATE.scans
<A> has <X>	Some $L \in \langle A \rangle$ has scans[L] == <X>
<A> has no <X>	No $L \in \langle A \rangle$ has scans[L] == <X>
<A> contains only <X>	<A> non-empty and every occupied $L \in \langle A \rangle$ has scans[L] == <X>
count in <A>	$\backslash \{ L \in \langle A \rangle : L \in \text{scans} \} \backslash $ (numeric)
count of <X> in <A>	$\backslash \{ L \in \langle A \rangle : \text{scans}[L] == \langle X \rangle \} \backslash $

Condition	Value
	(numeric)
count of empty slots in <A>	$\setminus \{L \in \langle A \rangle : L \text{ covered by visited, } L \notin \text{scans}\} $ (numeric)
empty slots in <A>	List of $L \in \langle A \rangle$ known empty
occupied slots in <A>	List of $L \in \langle A \rangle$ in scans
types in <A>	Set of scans[L] for $L \in \langle A \rangle$ in scans
count by type in <A>	Map $\{T \rightarrow \setminus \{L \in \langle A \rangle : \text{scans}[L] == T\} $

Cross-target.

Condition	Value
more empty room in <A> than 	count of empty slots in <A> > count of empty slots in
same count in <A> and <Z>	count in <A> == count in <Z>
types shared by <A> and <Z>	types in <A> $\cap$ types in <Z>
types in <A> not in <Z>	types in <A> $\setminus$ types in <Z>

Scene-wide (read across all of STATE.scans and STATE.visited).

Condition	Value
locations holding <X>	List of every $L \in \text{scans}$ with scans[L] == <X>
no location holds <X>	The above list is empty
first location holding <X> exists	Some $L \in \text{scans}$ has scans[L] == <X>
first empty location exists	Some L covered by visited is absent from scans
every observe pose visited	STATE.visited $\supseteq$ {observe poses in get_positions}

Composition. and, or, not, >, >=, <, <=, ==, +, -.

A composite condition (e.g. a and b) should be emitted split:

```
a = <value>
b = <value>
a and b = <value>
```

6. RESOLUTION

Every step emits a RESOLUTION block:

```
RESOLUTION
<resolution lines>
selected: <selected>
<resolution lines>
```

Resolution happens going top-down through the step defined in the cursor as described in §5.

6.1 Selected: options

The LLM first resolves what it will execute using selected: and then resolves everything else top-down

Step kind	Entry-phase lines
Linear	selected: linear
Branching	For each condition evaluated in head order, stopping at the first true (or after all if none true): one line <cond> = <value>. Then either selected: "<arm head text>" (the arm head text without trailing colon, in double quotes) for the first true arm, or selected: fallthrough if none matched.

7. TOOLS

```
TOOLS
move_to(observe_box1)
locate_shapes()
```

Each non-empty line is one tool call. Calls run in order; later calls in the same batch see effects of earlier ones.

An empty TOOLS block (just the header) is valid

8. OUTCOME and ANSWER

8.1 Outcome

Emitted in the same invocation as the step whose selected arm ends with answer_<K>.

```
OUTCOME
answer_<K>
```

8.2 Answer invocation

Triggered by cursor: answer_<K> or cursor: error in input STATE. The model emits combination O6:

```
STATE
cursor: answer_<K> | error
```

```
... (final field values, all tool calls applied that happened before the error one)
```

ANSWER

```
<natural-language reply>
```

The model composes prose by:

1. Looking up `answer_<K>` in the input PROGRAM to find its program location (which arm of which step terminates).
2. Reading STATE for the values needed in the reply.
3. Writing prose appropriate to the situation (success confirmation, refusal explanation, query response).

There are no tags. Register is determined by program context.

9. CHECK AND ERROR

After each tool call is executed, the model checks the result and emits either

```
CHECK
```

or

```
ERROR
```

10. Walkthroughs

10.1 Simple pick

User task: "Pick up a cube from box1"

Position list: `pick_cube_box1_1, pick_cube_box1_2, place_cube_box1_1, place_cube_box1_2 observe_box1, pick_cube_box2_1, pick_cube_box2_2, place_cube_box2_1, place_cube_box2_2 observe_box2`

Invocation 1 — I1 → O1

PRECALL

```
get_positions()
```

PRECALL RESULTS

PRECALL RESULTS

```
get_positions(): {  
  status: OK  
  content: pick_cube_box1_1, ...  
}
```

Invocation 2 — I2 → O3

CHECK

Invocation 3 — I1 → O2

```
MAP
cube = cube
box1 = [box1_1, box1_2]

STATE
cursor: step_1
position: unknown
gripper: unknown
held: unknown
scans: {}
visited: []
memory: {}

PROGRAM
STEP 1:
  check gripper
STEP 2:
  when gripper is closed:
    answer_1
  otherwise:
    observe box1
STEP 3:
  when box1 has no cube:
    answer_2
  otherwise:
    pick from first location in box1 holding cube
    answer_3

RESOLUTION
selected: linear

TOOLS
get_gripper_state()
```

TOOLS RESULTS

```
TOOLS RESULTS
get_gripper_state(): {
  status: OK
  content: open
}
```

Invocation 4 — I2 → O3

CHECK

Invocation 5 — I3 → O5

STATE

```
cursor: step_2
position: unknown
gripper: open
held: none
scans: {}
visited: []
memory: {}

RESOLUTION
gripper is closed = False
selected: otherwise
observe poses = [observe_box1]

TOOLS
move_to(observe_box1)
locate_shapes()
```

TOOLS RESULTS

```
TOOLS RESULTS
move_to(observe_box1): {
  status: OK
  content:
}
```

Invocation 6 — I2 → O3

CHECK

TOOLS RESULTS

```
TOOLS RESULTS
move_to(observe_box1): {
  status: OK
  content:
}
locate_shapes(): {
  status: OK
  content: { box1_2: cube}
}
```

Invocation 7 — I2 → O3

CHECK

Invocation 8 — I3 → O5

```
STATE
cursor: step_3
position: observe_box1
gripper: open
held: none
scans: { box1_2: cube}
```

```
visited: [observe_box1]
memory: {}

RESOLUTION
box1 has no cube = False
selected: otherwise
L = first location in box1 holding cube = box1_2
T = scans[box1_2] = cube

TOOLS
move_to(pick_cube_box1_2)
close_gripper()

OUTCOME
answer_3
```

TOOLS RESULTS

```
TOOLS RESULTS
move_to(pick_cube_box1_2): {
  status: ok
  content:
}
```

Invocation 9 — I2 → O3

CHECK

TOOLS RESULTS

```
TOOLS RESULTS
move_to(pick_cube_box1_2): {
  status: ok
  content:
}
close_gripper(): {
  status: ok
  content:
}
```

Invocation 10 — I2 → O3

CHECK

Invocation 11 — I3 → O6

```
STATE
cursor: answer_3
position: pick_cube_box1_2
gripper: closed
held: cube
scans: {}
visited: [observe_box1]
```



```
memory: {}
```

ANSWER

Picked up cube from box1_2.

10.2 Place a cube

In this task user has opened gripper and now hands a part to the robot and then asks it to place it.
Scene as in 10.1

User task: "You are holding a brick, put it somewhere"

Invocation 1 — I1 → O1

PRECALL

```
get_positions()
```

PRECALL RESULTS

PRECALL RESULTS

```
get_positions(): {  
  status: OK  
  content: pick_cube_box1_1, ...  
}
```

Invocation 2 — I2 → O3

CHECK

Invocation 3 — I1 → O2

MAP

```
brick = cube
```

STATE

```
cursor: step_1  
position: unknown  
gripper: closed  
held: brick  
scans: {}  
visited: []  
memory: {}
```

PROGRAM

STEP 1:

```
  observe everything
```

STEP 2:

```
  when first empty location exists:  
    place at first empty location  
    answer_1  
  otherwise:  
    answer_2
```

RESOLUTION

```
selected: linear
```

```
observe poses = [observe_box1, observe_box2]
```

```
TOOLS
move_to(observe_box1)
locate_shapes()
move_to(observe_box2)
locate_shapes()
```

TOOLS RESULTS

```
TOOLS RESULTS
move_to(observe_box1): {
  status: ok
  content:
}
```

Invocation 4 — I2 → O3

CHECK

TOOLS RESULTS

```
TOOLS RESULTS
move_to(observe_box1): {
  status: ok
  content:
}
locate_shapes(): {
  status: ok
  content: { box1_1: cube, box1_2: cube }
}
```

Invocation 5 — I2 → O3

CHECK

TOOLS RESULTS

```
TOOLS RESULTS
move_to(observe_box1): {
  status: ok
  content:
}
locate_shapes(): {
  status: ok
  content: { box1_1: cube, box1_2: cube }
}
move_to(observe_box2): {
  status: ok
  content:
}
```

Invocation 6 — I2 → O3

CHECK

TOOLS RESULTS

```
TOOLS RESULTS
move_to(observe_box1): {
  status: ok
  content:
}
locate_shapes(): {
  status: ok
  content: { box1_1: cube, box1_2: cube }
}
move_to(observe_box2): {
  status: ok
  content:
}
locate_shapes(): {
  status: ok
  content: { box2_1: cube }
}
```

Invocation 7 — I2 → O3

CHECK

Invocation 8 — I3 → O5

```
STATE
cursor: step_2
position: observe_box2
gripper: closed
held: brick
scans: {box2_1: cube}
visited: [observe_box1, observe_box2]
memory: {}

RESOLUTION
first empty location exists = true
selected: when first empty location exists
L = first empty location = box2_2
L = box2_2
X = held = brick

TOOLS
move_to(place_cube_box2_2)
open_gripper()

OUTCOME
answer_1
```

TOOLS RESULTS

TOOLS RESULTS

```
move_to(place_cube_box2_2): {  
  status: ok  
  content:  
}
```

Invocation 9 — I2 → O3

CHECK

TOOLS RESULTS

```
TOOLS RESULTS  
move_to(place_cube_box2_2): {  
  status: ok  
  content:  
}  
open_gripper(): {  
  status: ok  
  content:  
}
```

Invocation 10 — I2 → O3

CHECK

Invocation 11 — I3 → O6

```
STATE  
cursor: answer_1  
position: place_cube_box2_2  
gripper: open  
held: none  
scans: {box2_1: cube, box2_2: cube}  
visited: [observe_box1, observe_box2]  
memory: {}
```

```
ANSWER  
Placed the brick at the box2_2.
```

End of spec.